เส้นทางหยก (jade_path)



อาจารย์เมฆามอบตราหยกดวงแรกให้อรุณก่อนออกจากสำนักหยก ตราแต่ละดวงสลักรหัสของตนเองไว้ และเชื่อมไปยังตราดวงถัดไปด้วยสายพลัง อรุณต้องร้อยตราทั้งหมดตามลำดับรหัสที่ได้รับ จากนั้นตามสายพลังจากตราแรกถึงตราสุดท้ายเพื่อบันทึกเส้นทาง และบางครั้งต้องค้นหาว่าตราที่มีรหัสหนึ่ง ๆ อยู่ลำดับที่เท่าไรบนเส้นทาง

สิ่งที่ฝึกในด่านนี้: สร้างและเดินลิสต์ (singly linked list): `new`, `head`, `nullptr`, การเดินด้วย $p = p$->`next`

รายละเอียดการเขียนโปรแกรม

เขียนฟังก์ชันที่ประกาศในไฟล์ `jade_path.h` ต่อไปนี้ โดยไม่ต้องเขียน `main` และไม่ต้องอ่านหรือพิมพ์ข้อมูลทางอินพุตและเอาต์พุตมาตรฐาน (โปรแกรมตรวจของผู้จัดทำหน้าที่นั้น)

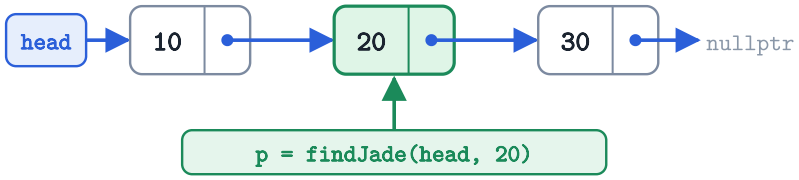
```
struct Node {
    int data;
    Node* next;
};
Node* buildPath(const std::vector<int>& values);
std::vector<int> readPath(Node* head);
Node* findJade(Node* head, int x);
```

- `buildPath(values)`: สร้างลิสต์โยงเดี่ยวด้วย `new` โดยโหนดที่ i มี `data` เท่ากับ `values[i]` เชื่อมกันตามลำดับเดียวกับ `values` แล้วคืนพอยน์เตอร์ของโหนดแรก ถ้า `values` ว่างให้คืน `nullptr` โหนดสุดท้ายต้องมี `next` เป็น `nullptr`
- `readPath(head)`: คืนเวกเตอร์ของค่า `data` ตั้งแต่โหนดแรกถึงโหนดสุดท้าย (ลิสต์ว่างคืนเวกเตอร์ว่าง) ห้ามเปลี่ยนแปลงลิสต์
- `findJade(head, x)`: คืนพอยน์เตอร์ของโหนดแรกสุด (ใกล้หัวลิสต์ที่สุด) ที่มี `data == x` ถ้าไม่พบให้คืน `nullptr` ห้ามเปลี่ยนแปลงลิสต์
- โหนดที่ `buildPath` สร้างต้องอยู่บน heap (สร้างด้วย `new`) ลิสต์ต้องเป็นสายตรง ไม่มีวงวน และท้ายลิสต์ชี้ `nullptr`
- `readPath` และ `findJade` ห้ามเปลี่ยนลิสต์ — โปรแกรมตรวจเทียบ "ตัวโหนด" ทุกตัวก่อนและหลังเรียก
- `findJade` ต้องคืนโหนดที่อยู่ในลิสต์นั้นจริง ๆ ไม่ใช่โหนดใหม่ที่สร้างขึ้นมาเอง

ลิสต์ที่ได้จาก `buildPath({10, 20, 30})`



`findJade(head, 20)` คืนพอยน์เตอร์ของโหนดที่อยู่ตำแหน่ง 1 (นับจาก 0)



รูปที่ 1 ลิสต์โยงเดี่ยว: `head` ชี้โหนดแรก แต่ละโหนดเก็บ `data` กับ `next` และโหนดสุดท้ายชี้ `nullptr`

การส่งงาน: ดาวน์โหลดไฟล์แนบ `jade_path.cpp` (ไฟล์โครง) และ `jade_path.h` จากหน้าโจทย์ แล้วเขียนฟังก์ชันลงในไฟล์ `jade_path.cpp` ให้เสร็จ จากนั้นส่งเฉพาะไฟล์ `jade_path.cpp` ห้ามแก้ไขไฟล์ `jade_path.h` ระบบจะคอมไพล์ไฟล์ของคุณพร้อมกับ `grader.cpp` ของผู้จัด ไฟล์ `attachment.zip` มีโปรแกรมตรวจสำหรับทดลองในเครื่อง (`g++ -std=c++17 -O2 grader.cpp jade_path.cpp -o jade_path`) ซึ่งตรวจการใช้ `new` / `delete` และความถูกต้องของโหนดให้เหมือนกับระบบจริง

ตัวอย่างการเรียกฟังก์ชัน

การเรียกฟังก์ชัน	ผลที่ได้
`buildPath({10, 20, 30})`	ได้ลิสต์ $10 \to 20 \to 30$ และ `readPath` คืน `{10, 20, 30}`
`buildPath({})`	คืน `nullptr` และ `readPath(nullptr)` คืน `{}`
`findJade(head, 20)` เมื่อลิสต์คือ $10 \to 20 \to 30$	คืนโหนดตำแหน่ง 1
`findJade(head, 8)` เมื่อลิสต์คือ $4 \to 8 \to 8 \to -2$	คืนโหนดตำแหน่ง 1 (ตัวแรกที่พบ)
`findJade(head, 5)` เมื่อลิสต์คือ $10 \to 20 \to 30$	คืน `nullptr`

ตารางแสดงตัวอย่างการเรียกที่แยกจากกัน ตำแหน่งของโหนดนับจาก 0 โดยโหนดแรกคือตำแหน่ง 0

ข้อจำกัด

- $1 \le Q \le 100$
- $0 \le N \le 200\ 000$ และ $0 \le K \le 1\ 000$ ในแต่ละชุด
- ผลรวมของ N ทุกชุดในกรณีทดสอบเดียวกันไม่เกิน 300 000 และผลรวมของ $N \times K$ ไม่เกิน 10^7
- $-10^9 \le \texttt{data}, x \le 10^9$ (ค่าซ้ำและค่าติดลบเป็นข้อมูลปกติ)
- เวลา 2 วินาที และหน่วยความจำ 256 MiB ต่อกรณีทดสอบ

ปัญหาย่อย

1. (20 คะแนน) $1 \le N \le 10$ และ $K = 0$ ทุกชุด (ยังไม่ต้องใช้ `findJade`)
2. (30 คะแนน) $N \le 1\ 000$ และ $K \le 1\ 000$ ทุกชุด
3. (50 คะแนน) ไม่มีข้อจำกัดเพิ่มเติม

แต่ละปัญหาย่อยคิดคะแนนแยกกัน จะได้รับคะแนนของปัญหาย่อยเมื่อผ่านกรณีทดสอบทุกกรณีในปัญหาย่อยนั้น

รูปแบบข้อมูลสำหรับโปรแกรมตรวจตัวอย่าง

บรรทัดแรก: จำนวนเต็ม Q คือจำนวนชุดข้อมูล
แต่ละชุดมี 4 บรรทัด: (1) จำนวนเต็ม N (2) ค่า N ค่าที่ใช้เรียก `buildPath` ตามลำดับ (บรรทัดว่างถ้า $N = 0$) (3) จำนวนเต็ม K (4) ค่า x จำนวน K ค่า (บรรทัดว่างถ้า $K = 0$)
โปรแกรมตรวจเรียก `buildPath` หนึ่งครั้ง แล้วเรียก `readPath` หนึ่งครั้ง และเรียก `findJade` ทีละคำค้นรวม K ครั้ง จากนั้นพิมพ์ 2 บรรทัดต่อหนึ่งชุด: บรรทัดแรกคือขนาดของเวกเตอร์ที่ `readPath` คืน ตามด้วยค่าทั้งหมดของเวกเตอร์ บรรทัดที่สองคือผลของ `findJade` แต่ละคำค้น โดยพิมพ์ตำแหน่งของโหนดที่ได้ (นับจาก 0) หรือ -1 ถ้าได้ `nullptr`

ตัวอย่างข้อมูลนำเข้าและผลลัพธ์

ข้อมูลนำเข้า 1	ผลลัพธ์ 1
2 4 10 20 30 20 3 20 99 30 0 0	4 10 20 30 20 1 -1 2 0

ชุดแรกมี 4 โหนด ค่า 20 ปรากฏสองครั้ง `findJade` ต้องคืนตัวแรกคือตำแหน่ง 1 ส่วน 99 ไม่พบจึงได้ -1 และ 30 อยู่ตำแหน่ง 2 · ชุดที่สองเป็นลิสต์ว่าง
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 2, 3

ข้อมูลนำเข้า 2	ผลลัพธ์ 2
3 1 5 0 2 -7 3 0 3 1000000000 -1000000000 0 0	1 5 2 -7 3 3 1000000000 -1000000000 0

ไม่มีคำค้นเลย $(K = 0)$ จึงตรวจเฉพาะการสร้างและการอ่านลิสต์ รวมถึงค่าติดลบและค่าสุดขอบ
ตัวอย่างนี้สอดคล้องกับปัญหาย่อยที่ 1, 2, 3